



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG



HARBORFLOW LOGISTICS AB

Assignment 1

HarborFlow Dispatch Console

A business-critical Python product for urban logistics

COURSE DIT014 - Fundamentals of Programming | HT26 | Group assignment: 3-4 students

Teacher: Björn Praestegaard Larsen, Lialia Beniaminova

Version date: 4 August 2026

Your commission



HarborFlow Logistics AB coordinates same-day parcel deliveries between the Port of Gothenburg, local businesses and neighborhood pickup points. The company has grown faster than its spreadsheets. Dispatchers now lose time checking booking references, calculating quotes, balancing van capacity and explaining late-delivery risks to customers.

Your team has been hired to deliver a terminal-based Dispatch Console. It must feel like one coherent business product: every feature is available from a persistent menu, every output follows the contract exactly, and the program returns safely to the menu after completing a service.

TEAM CONTRACT Complete this assignment in a CodeGrade group of 3-4 students. The submitted code is a shared deliverable, but every member must understand and be able to explain the complete system.

Learning goals

- Translate a client need into precise inputs, decisions, state changes and outputs.
- Use printing, input, variables, numeric expressions and string operations correctly.
- Design conditionals, for loops, while loops, lists and functions that work together.
- Test normal, boundary and invalid cases against an exact interface contract.
- Organize a multi-feature Python program so another developer can maintain it.

Permitted technical scope

Use Python built-ins and the math library only. Do not use dictionaries, files, regular expressions, list comprehensions, lambda expressions, classes, third-party packages or `exit()`. Avoid shortcuts that bypass the concepts being assessed.

Assessment and delivery standard

Result	Assessment decision
G - Pass	All Tasks 1-9 satisfy the stated behavior, exact interface contract, technical constraints and handover requirements. The integrated product is readable, tested and explainable by every team member.
U - Fail	One or more compulsory requirements are missing, incorrect, unreliable, outside the permitted scope or cannot be explained as a shared team deliverable.

ONE PASS STANDARD The assignment carries no grades 3, 4 or 5. Every task is compulsory. CodeGrade evidence and the examiner's holistic review together determine G or U.

CodeGrade submission

STARTER PACKAGE Download and extract the Assignment 1 starter package from the Canvas assignment before coding. It contains `harborflow_app.py`, a readme template and a test ledger. Assignment 1 uses console input and requires no external dataset.

- Create a CodeGrade group containing exactly 3-4 students and use the naming format specified in Canvas.
- Submit `harborflow_app.py` and `readme.txt`. Upload any additional Python modules only if your design uses them.
- The readme must list each member and describe concrete contributions. 'Everyone did everything' is not sufficient.
- CodeGrade uses exact automated tests. Prompts, punctuation, capitalization, spacing, order and filenames are part of the product contract.
- You may submit repeatedly before the deadline. The final recorded submission is assessed.

Definition of done

- The program starts at the main menu and exits only through menu option 1.
- Every required task is accessible, runs independently and returns to the menu.
- Functions receive meaningful arguments and return values where appropriate.
- No global variable is used merely to avoid passing data between functions.
- The team has tested the reference examples plus boundary cases.

Task 1: Build the dispatch menu

BUSINESS OUTCOME Give dispatchers one dependable entry point to every service.

When `harborflow_app.py` starts, print the exact menu below. Read one integer option. Option 1 ends the program by changing loop state; `exit()` is not allowed. After Tasks 2-7 finish, the program returns to this menu.

```
HARBORFLOW DISPATCH CONSOLE
1. Close console
2. Validate booking reference
3. Calculate delivery quote
4. Consolidate parcel labels
5. Check van capacity
6. Classify service performance
7. Produce weekly dispatch report
Select service:
```

Required behavior

- Use a while loop to keep the console active.
- Use conditional logic to call the correct task function.
- For the required services, CodeGrade supplies an integer from 1 to 8.
- When option 1 is selected, print the closing message exactly once.

```
Console closed. Dispatch data remains safe.
```

ENGINEERING NOTE Create the menu first, but keep feature logic in functions. A menu branch should call a function, not contain an entire task implementation.

Task 2: Validate a booking reference

BUSINESS OUTCOME Prevent mistyped references from entering customer and warehouse workflows.

A valid HarborFlow reference has the business format HFL-CCC-NNNN. HFL is fixed, CCC is a three-letter customer code and NNNN is a four-digit shipment number. The user may type lowercase letters and spaces around the input. Normalize the reference, test every rule and print the result.

Rules

- Remove leading and trailing spaces and convert letters to uppercase.
- The normalized reference must contain exactly 12 characters.
- Positions 4 and 8 must contain hyphens.
- The customer code must contain three letters; the shipment part must contain four digits.
- Do not use regular expressions.

Example A

```
Booking reference:  hfl-nor-2048
Valid reference: HFL-NOR-2048
```

Example B

```
Booking reference: HFL-N4R-2048
Invalid booking reference.
```

EXPECTED DESIGN Implement a function such as `validate_reference(reference)` that returns the normalized reference when valid and an empty string when invalid.

Task 3: Calculate a delivery quote

BUSINESS OUTCOME Give sales staff a consistent quote before they promise a price to a customer.

Ask for distance in kilometres, total parcel weight in kilograms and a service code: S for Standard, X for Express or P for Priority. Calculate the quote from the rules below. Inputs are positive numbers and a valid service code for the required task.

Component	Business rule
Base charge	45.00 SEK
Distance	6.50 SEK per kilometre
Weight	4.00 SEK per kilogram
Standard	Multiplier 1.00
Express	Multiplier 1.25
Priority	Multiplier 1.60

Formula

```
subtotal = 45.00 + distance * 6.50 + weight * 4.00
quote = subtotal * service_multiplier
```

Print the quote with exactly two decimal places.

```
Distance (km): 12.5
Weight (kg): 8
Service code: X
Delivery quote: 197.81 SEK
```

BOUNDARY THINKING Your function should not contain input() calls if a cleaner design can pass distance, weight and service code as arguments.

Task 4: Consolidate parcel labels

BUSINESS OUTCOME Turn duplicate scan data into a clean loading list without losing operational order.

A scanner exports parcel labels as one comma-separated line. Normalize each label by removing surrounding spaces and converting it to uppercase. Build a list containing each distinct label once, preserving the order in which it first appeared.

The label format is not validated in this task; focus on splitting, normalization, loops, membership checks and ordered output. Do not use set().

Example

```
Scanned labels: gb-104, GB-220, gb-104, se-011, GB-220
Unique load list:
1. GB-104
2. GB-220
3. SE-011
Total unique parcels: 3
```

Required behavior

- Split the original string at commas.
- Use a loop to normalize and evaluate every scanned item.
- Preserve the first-seen order.
- Use a loop to print numbered output; do not print the Python list representation.

CLIENT EXPECTATION The printed order matters because warehouse staff load parcels in the order shown by the console.

Task 5: Check van capacity

BUSINESS OUTCOME Help a dispatcher accept as many parcels as possible without overloading a vehicle.

Read a van capacity and a comma-separated sequence of parcel weights. Process weights from left to right. Accept a parcel when it fits within the remaining capacity. If it does not fit, reject it and continue checking later parcels. All weights are positive numbers.

Print one decision per parcel, then print the accepted count, loaded weight and remaining capacity with two decimals.

Example

```
Van capacity (kg): 100
Parcel weights (kg): 40, 65, 20, 35
Parcel 1: ACCEPTED
Parcel 2: REJECTED
Parcel 3: ACCEPTED
Parcel 4: ACCEPTED
Accepted parcels: 3
Loaded weight: 95.00 kg
Remaining capacity: 5.00 kg
```

Important interpretation

Rejecting one parcel does not stop the process. A later, lighter parcel may still fit. Your algorithm therefore needs both accumulated state and a decision inside the loop.

TEST THIS Capacity 50 with weights 50; capacity 50 with weights 51,49; and a sequence where several rejected parcels appear before one accepted parcel.

Task 6: Classify service performance

BUSINESS OUTCOME Translate operational numbers into a consistent customer-service status.

For one route, ask for promised minutes, actual minutes and damaged parcels. Calculate $\text{delay} = \text{actual} - \text{promised}$. Use the decision table in the stated order.

Condition	Status
damaged parcels > 0	SERVICE FAILURE
otherwise, delay <= 0	ON TIME
otherwise, delay <= 15	MINOR DELAY
otherwise	MAJOR DELAY

Damage has priority over timing. A route that arrives early with damaged parcels is still a service failure.

Examples

```
Promised minutes: 90
Actual minutes: 102
Damaged parcels: 0
Delay: 12 minutes
Service status: MINOR DELAY

Promised minutes: 90
Actual minutes: 80
Damaged parcels: 1
Delay: -10 minutes
Service status: SERVICE FAILURE
```

BOUNDARY CASES Test delay values -1, 0, 1, 15 and 16, and repeat a timing case with one damaged parcel.

Task 7: Produce the weekly dispatch report

BUSINESS OUTCOME Give operations management evidence for a Monday performance meeting.

Read seven comma-separated integers representing completed deliveries from Monday through Sunday. Then read the weekly target as an integer. Calculate and print the total, average, highest day, lowest day and number of days meeting or exceeding the target.

For ties, report the last day with the highest or lowest value. Do not use `max()`, `min()`, `index()`, `sum()` or statistics libraries; implement the traversal yourself.

Example

```
Completed deliveries: 42, 55, 55, 38, 61, 61, 20
Daily target: 50
Weekly dispatch report
Total deliveries: 332
Average per day: 47.43
Highest day: Saturday (61)
Lowest day: Sunday (20)
Days meeting target: 4
```

Expected structure

- Store the seven values in a list.
- Use accumulators and comparison state while traversing the list.
- Map positions to weekday names with a second list.
- Use functions to separate calculation from printing where practical.

Task 8: Make the console resilient

BUSINESS OUTCOME Keep the dispatch desk operational when a user enters a value outside the agreed range.

Add validation loops for the cases below. After an error, repeat only the affected prompt. This compulsory task focuses on invalid values and formats and must be integrated across the product.

- Menu: accept only integers 1-8.
- Quote: distance and weight must be greater than zero; service code must be S, X or P.
- Capacity: capacity and every parcel weight must be greater than zero.

- Performance: promised and actual minutes cannot be negative; damaged parcels cannot be negative.
- Weekly report: exactly seven non-negative delivery counts are required; target must be non-negative.

Required error templates

```
Error - Select a service from 1 to 8.
Error - Value must be greater than zero.
Error - Service code must be S, X or P.
Error - Weekly report requires 7 delivery counts.
```

The program must not crash, silently replace an invalid value, or return to the main menu before valid input has been collected.

Task 9: Compare delivery scenarios

BUSINESS OUTCOME Help a sales coordinator compare Standard, Express and Priority quotes before presenting options to a customer.

Add menu option 8: Compare service scenarios. Read one distance and one weight. Reuse your quote calculation function to calculate all three service prices. Print each option and identify the cheapest and most expensive service. Update the allowed menu range and closing behavior accordingly.

```
Distance (km): 18
Weight (kg): 12
Service comparison
Standard: 210.00 SEK
Express: 262.50 SEK
Priority: 336.00 SEK
Cheapest service: Standard
Most expensive service: Priority
```

Technical expectations

- Do not copy the quote formula three times.
- A single calculation function must accept the service code or multiplier.
- Keep the printed order Standard, Express, Priority.
- Format every price with exactly two decimal places.
- After Task 8 is implemented, validate distance and weight before calculating.

PASS-LEVEL DESIGN Correct output is necessary but not sufficient. The integrated code must show meaningful function boundaries, reuse and separation of concerns.

Engineering the product as a team

The client receives one product, not four unrelated code fragments. Divide work to create ownership while using reviews and integration sessions to maintain shared understanding.

Recommended workflow

1. Agree on the exact interface, filenames and shared function conventions before coding.
2. Assign initial task ownership, but pair-review every completed feature.
3. Integrate frequently into one runnable console; do not wait until the final day.
4. Maintain a shared test ledger with input, expected result, actual result and owner.
5. Rotate the person who runs the integrated demonstration so every member can explain the system.

Suggested architecture

```
def main():
    # maintain menu state and dispatch to feature functions

def validate_reference(reference):
    # return normalized reference or an empty string

def calculate_quote(distance, weight, service_code):
    # return the calculated price

if __name__ == "__main__":
    main()
```

STARTER, NOT A SOLUTION You may adapt these function boundaries. The team remains responsible for designing arguments, return values, test cases and complete behavior.

Acceptance test campaign

Before submission, the team must be able to demonstrate the following evidence. CodeGrade tests additional cases.

Area	Minimum evidence
Menu	Every option works; each feature returns; option 1 closes once
Reference	Valid lowercase input, wrong length, misplaced hyphen, digit in customer code
Quote	All service codes and a decimal-value case
Labels	Duplicates, mixed case, surrounding spaces, one item
Capacity	Exact fit, first rejection, later item fits, all rejected
Performance	All boundaries and damage precedence
Weekly report	Typical week, all equal, last-day tie, target met zero/seven days

CodeGrade preflight

- Place harborflow_app.py, readme.txt and any required modules in a clean folder, then run it from the command line without an IDE-specific configuration.
- Compare prompts and outputs character by character with this specification.
- Remove debugging prints and test-only input values.
- Upload the exact files, confirm CodeGrade recorded the group attempt, review feedback and resubmit before the deadline when allowed.

Submission checklist and client handover

Required files

File	Contents
harborflow_app.py	Integrated executable Python product with main()
readme.txt	Group members, concrete task division, design notes and known limitations
Additional .py files	Only when used by the submitted product; include every dependency

Final quality review

- All group members are listed in CodeGrade and readme.txt.
- All compulsory Tasks 1-9 are complete and the applicable services are accessible from the menu.
- The program uses only permitted concepts and libraries, with understandable names, functions, indentation and comments.
- Reference examples and boundary tests have been rerun after final integration.
- The team has considered accessibility, avoided exposing unnecessary customer data and avoided wasteful repeated work.
- The final CodeGrade submission is recorded under the correct group.